Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- PPM'in tekrarı ve order-N modeli (ekrandan, 00:10:00)
  - PPM, J. Cleary ve I. Witten tarafından ortaya atılmış, A. Moffat tarafından iyileştirilmiştir. Slaytta adı "Prediction with Partial String Matching" diye geçer; order-N Markov modelini kullanan içerik (context) tabanlı bir sıkıştırma yöntemidir.
  - Klasik istatistiksel yöntemde bir sembolün olasılığı, metinde geçme sayısının toplam sembol sayısına oranıdır (örneğin q için $34/1217$). PPM'de ise olasılık, sembolün geçmişindeki N sembole (context'e) bağlı olarak verilir. Örneğin h ve t'nin olasılıkları ayrı ayrı başkadır, "th" ikilisinin olasılığı çok daha farklıdır.
  - Statik içerik tabanlı modelde context'ler ve olasılıkları önceden bir derlemden (korpus) çıkarılır ve kodlar sabittir. Adaptif (dinamik) modelde hazır tablo yoktur; olasılıklar ve kodlar doküman işlenirken sürekli değişir (dinamik Huffman gibi).
- Lipogram (ekrandan, 00:17:45)
  - Slayttaki tanıma göre bir metindeki karakterlerin olasılık değerleri birbirinden çok ayrıksa (varyans büyükse) bu duruma lipogram denir. Böyle bir doküman iyi sıkıştırılmaz.
  - Örnekler: Ernest Vincent Wright'ın "Gadsby" romanında E harfi yoktur. Walter Abish'in "Alphabetical Africa" romanı 52 bölümdür: 1. bölümde bütün kelimeler a ile, 2. bölümde a ve b ile başlar, 26. bölümde a-z ile başlar; kalan 26 bölümde aynı şekilde azalarak devam eder.
- Order-N Adaptive Context-Based Model (ekrandan, 00:20:00)
  - Input stream'den gelen S sembolü okunur, order-N modeline göre kendisinden önce gelen N sembol bulunur ve S'nin olasılığı tahmin edilir. Context bulunamazsa N bir azaltılır (switch).
  - Slayta göre teoride N çok büyük seçilir (N = 20.000), ama pratikte dezavantajı vardır: ilk 20.000 karakter sıkıştırılmadan (ASCII olarak) kodlanır ve olası context sayısı çok büyür.
  - 7 bitlik semboller için 128 farklı sembol vardır; order 2'de $128^2 = 16.384$, order 3'te $128^3 = 2.097.152$ farklı kombinasyon gelebilir.
  - Hocaya göre akademik çalışmalarda N en fazla 5, çoğunlukla 3'e kadar alınır. Bölümleri farklı konuları anlatan bir tarih kitabında büyük N ile çakışmalar azalır ve sonuç iyi olmaz.
- Order-4 context tablosu: «xyzzxyxyzzx» (ekrandan, 00:30:00 ve 00:40:00)
  - Tabloda Order 4, 3, 2, 1 ve 0 kolonları vardır; aranan context bulunamazsa alt order'a switch edilir. Hoca tabloyu sembol sembol, kalemle sayarak kurdu.
  - Order 0: x 4, y 3, z 4. Order 1: x→y 3; y→z 2, y→x 1; z→z 2, z→x 2. Order 2: xy→z 2, xy→x 1; yz→z 2; zz→x 2; zx→y 1; yx→y 1. Order 3: xyz→z 2; yzz→x 2; zzx→y 1; zxy→x 1; xyx→y 1; yxy→z 1. Order 4: xyzz→x 2; yzzx→y 1; zzxy→x 1; zxyx→y 1; xyxy→z 1; yxyz→z 1 (python ile doğrulandı).
  - Bir z daha gelince: Order 0'da z 5 olur; yeni context'ler x→z 1, zx→z 1, zzx→z 1, yzzx→z 1 eklenir.
  - Frekanslar çıkarıldıktan sonra olasılıklar hesaplanır ve semboller aritmetik kodlamadaki gibi $[0, 1)$ aralığına dağıtılarak kodlanır.
- Switch ve escape sembolü (ekrandan, 00:50:00)
  - PPM, sıfır olasılıklı bir context geldiğinde onu daha kısa bir context ile değiştirir. Bu Order 0'a kadar sürer. Order 0'da da yoksa sembol yenidir ve ona sabit bir olasılık

($1/$alfabe boyu) verilir.
  - Kod çözücünün nerede switch edildiğini bilmesi için **escape** sembolü kullanılır. Kodlayıcı switch etmeden önce escape'in olasılığını aritmetik kodlayıcıya gönderir. Her context bloğunun kendi escape'i vardır.
- Order-2 tablo ve escape frekansları: «assanissimassa» (ekrandan, 00:50:00)
  - Order 2: as→s 2/3, esc 1/3; ss→a 2/5, ss→i 1/5, esc 2/5; sa→n 1/2, esc 1/2; an→i, ni→s, is→s, si→m, im→a, ma→s her biri 1/2, esc 1/2.
  - Order 1: a→s 2/5, a→n 1/5, esc 2/5; s→s 3/9, s→a 2/9, s→i 1/9, esc 3/9; n→i 1/2, esc 1/2; i→s 1/4, i→m 1/4, esc 2/4; m→a 1/2, esc 1/2.
  - Order 0: a 4/19, s 6/19, n 1/19, i 2/19, m 1/19, esc 5/19.
  - Escape frekansı, o context'i izleyen farklı sembol sayısıdır (örneğin Order 0'da 5 farklı sembol, esc 5).
- Context «sa»'dan sonra gelen sembolün kodlanması (ekrandan, 01:00:00)
  - n gelirse: sa→n var, $P = 1/2$, 1 bit.
  - s gelirse: sa context'inde s yok; esc $1/2$, sonra a→s $2/5$. Toplam $-\log_2(1/2 \cdot 2/5) \approx 2{,}32$ bit.
  - m gelirse: esc $1/2$, a context'inde m yok, esc $2/5$, Order 0'da m $1/19$. Toplam $\approx 6{,}57$ bit.
  - d gelirse: esc $1/2$, esc $2/5$, Order 0'da da yok, esc $5/19$; Order -1'de $1/28$ (alfabe 28 sembol). Toplam $\approx 9{,}05$ bit; ASCII ile 8 bit olurdu (python ile doğrulandı).
- PPM'in versiyonları (ekrandan, 01:08:50)
  - Derste gösterilen yöntem PPMC'dir ve en çok kullanılandır. Versiyonlar yalnız escape sembolünün kullanımında ayrılır; sıkıştırma oranları arasında çok küçük farklar vardır.
  - PPMA: toplam frekans n ise escape'e $1/(n+1)$, diğer sembollere x/n olasılığı atanır.
  - PPMB: C context'i içinde s iki kez göründükten sonra C'yi izleyen S'ye olasılık atanır. «abc» context'i 3 kez, onu izleyen x 2 kez, y 1 kez görünmüşse x için $(2-1)/3$, y için $(1-1)/3$ atanır.
- Context'lerin trie ile tutulması (ekrandan, 01:10:00)
  - Context'lere hızlı erişmek için TRIE ağacı kullanılır: root'tan yaprağa giden her yol bir context'tir, düğümlerde sembol ve frekans durur. Ağacın derinliği order ile sınırlıdır.
  - Slayttaki örnek «zxzyzxxyzx» için on adımda kurulan trie'lerdir; her adımda frekansı artan düğümler yuvarlak içine alınır.
  - Switch işlemini kolaylaştırmak için düğümlere, bir kısa context'in düğümünü gösteren pointer'lar eklenir (örneğin «as» düğümünden «s» düğümüne).
- Sözlük tabanlı yöntemlere giriş
  - İstatistiksel yöntemlerde verinin istatistiksel modeli kullanılır. Sözlük tabanlı yöntemlerde olasılık ve değişken uzunluklu prefix kodlar yoktur; string'ler (bir ya da daha fazla karakter, anlamlı olması gerekmez) bir sözlükte toplanır ve token'larla kodlanır.
  - Sözlük statik (sabit) ya da dinamik (ekleme, silme, yeniden kurma serbest) olabilir.
- Statik sözlük örneği (ekrandan, 01:40:00 ve 01:48:30)
  - İngilizce dokümanlar için kelimelerden oluşan bir sözlük düşünülebilir; kelimeler ek aldığı için Türkçe gibi sondan eklemeli diller için uygun değildir.
  - Yarım milyon kelimelik sözlükte index için 19 bit gerekir ($2^{19} = 524.288$). Sözlükte olan kelime: 1 flag bit (0) + 19 bit = 20 bit. Olmayan 5 karakterlik kelime: 1 flag bit (1) + 7 bit uzunluk + 5×8 bit = 48 bit.
  - Soru: kelimelerin ne kadarı sözlükte olmalı ki sıkıştırma olsun? $N[20P+48(1-P)] = 40N \Rightarrow 28P = 8 \Rightarrow P \approx 0{,}29$. Kelimelerin %29'undan fazlası sözlükte olmalıdır.
- String sıkıştırma (ekrandan, 01:50:00 ve 01:53:40)
  - Sembollerden oluşan string'lere dayalı yöntemler tek bir sembole dayalı yöntemlerden daha etkilidir.
  - a_1 (0,8) ve a_2 (0,2) alfabesinde string boyu 1, 2, 3 için olasılık varyansı $0{,}6$; $0{,}78$; $0{,}792$, ortalama kod uzunluğu 1; $0{,}78$; $0{,}728$ bit/sembol olur (python ile doğrulandı).
- LZ ve türev algoritmalar (ekrandan, 01:57:10)
  - Jacob Ziv ve Abraham Lempel 1970'li yıllarda LZ77 ve LZ78'i geliştirdi. Türevleri: LZSS (Storer-Szymanski), LZFG (Fiala-Greene), LZRW1 (Ross Williams), LZW (Welch), LZMW (Miller-Wegman), LZAP (All prefix), LZY (Yabba), LZP (Prediction).

  - Türevler; pencere boyutu, arama biçimi (linear ya da circular), alan uzunlukları gibi parametrelerle ayrılır. En çok kullanılanı LZW'dir.
- LZ77, kayan pencere (ekrandan, 02:00:00)
  - Ana fikir, input stream'den daha önce okunmuş bilgiyi sözlük olarak kullanmaktır. Pencere iki parçadır: solda **arama alanı (search buffer)**, yani en son kodlanan semboller; sağda **kaynak kod alanı (look-ahead buffer)**, yani henüz kodlanmamış semboller. Arama alanı binlerce, kaynak kod alanı onlarca byte'tır.
  - Kodlayıcı, look-ahead'in ilk sembolünü arama alanında sağdan sola arar ve en uzun çakışmayı bulmaya çalışır. Token üç alanlıdır: (offset, length, sonraki sembol).
  - Slayttaki örnekte «…easily t» arama alanında, «eases …» look-ahead'dedir: e önce «easily»de (offset 8), sonra «eastman»da bulunur; «eas» 3 sembolle eşleşir. En son bulunan çakışma referans alınır: token (16, 3, «e») (ekrandan, 02:10:00).
- LZ77 başlangıç örneği ve alan uzunlukları (ekrandan, 02:14:50)
  - Arama alanı boşken «sir sid eastman»: (0,0,«s»), (0,0,«i»), (0,0,«r»), (0,0,«␣»), (4,2,«d»), (4,1,«e»), (0,0,«a»). Hoca devamını sözlü olarak (10,1,«t») diye verdi. (0,0,«s») ile yalnız bir sembol kodlanır, bu iyi sıkıştırma sağlamaz.
  - Arama alanı uzunluğu S ise offset $\lceil \log_2 S \rceil$ bit (pratikte 10-12 bit); look-ahead uzunluğu L ise length $\lceil \log_2 (L-1) \rceil$ bit; sembol alanı 8 bit. Bir token yaklaşık $11+5+8 = 24$ bittir.
  - Kod çözücü kodlayıcıdan daha basittir; bu yüzden bir kez sıkıştırılıp çok kez açılan uygulamalar (arşiv) için elverişlidir.
- LZ77'nin geliştirilmeleri (ekrandan, 02:20:00)
  - Offset ve length için farklı uzunluklar, farklı buffer boyutları denenmiştir. Search buffer büyütülünce çakışmalar iyileşmiş ama arama süresi artmıştır.
  - Linear window yerine **circular queue** kullanılmıştır: pencere dolunca yeni sembol en eskinin yerine yazılır ve start/end pointer'ları kaydırılır.
- LZ78 (ekrandan, 02:30:00)
  - Kayan pencere, arama alanı ve kaynak kod alanı yoktur. Boş başlayan ve hafızayla sınırlı bir sözlük vardır. Token iki alanlıdır: (çakışan string'in sözlükteki sırası, sonraki sembol).
  - «sir sid eastman easily teases sea sick seals» için sözlük: 1 «s» (0,«s»); 2 «i» (0,«i»); 3 «r» (0,«r»); 4 «␣» (0,«␣»); 5 «si» (1,«i»); 6 «d» (0,«d»); 7 «␣e» (4,«e»); 8 «a» (0,«a»); 9 «st» (1,«t»); 10 «m» (0,«m»); 11 «an» (8,«n»); 12 «␣ea» (7,«a»); 13 «sil» (5,«l»); 14 «y» (0,«y») (python ile doğrulandı).
  - Sözlük trie ile tutulur; root'tan alfabedeki bütün karakterler çıkar. Sözlük dolunca: sözlük dondurulur (frozen), tamamen silinip yeniden kurulur ya da en az kullanılan string silinip yerine yenisi yazılır.
- LZW (ekrandan, 02:50:00 ve 03:00:00)
  - Codebook (sözlük) dinamik oluşturulur ve önce alfabedeki sembollerle başlatılır. Codebook'un boyu verilebilir (8, 16, 32 code word); verilmezse input bitene kadar büyür. Kod uzunluğu codebook boyuna bağlıdır (16 kayıt için 4 bit, 32 için 5 bit).
  - Kural: en uzun eşleşen string'in kodu output stream'e yazılır, bu string'e sonraki sembol eklenerek codebook'a yazılır, kodlamaya o sonraki sembolden devam edilir.
  - Örnek «abbbaacbbabbaacacaabb», alfabe a, b, c: codebook 3 ab, 4 bb, 5 bba, 6 aa, 7 ac, 8 cb, 9 bbab, 10 bbaa, 11 aca, 12 acaa, 13 abb. Output stream: 0 1 4 0 0 2 5 5 7 11 3 1 (python ile doğrulandı).
  - Kod çözücü de alfabeyle başlar ve kodları okurken aynı codebook'u yeniden kurar.

Hocanın Özellikle Vurguladığı Kısımlar

- PPM'de N'in seçimi
  - N çok büyük alınırsa ilk N karakter sıkıştırılmadan gider, kombinasyon sayısı patlar ve bir kez geçen context'lerle boşuna uğraşılır. Pratikte 3, en fazla 5.
- Escape sembolünün gerekçesi
  - Kodlayıcı switch ettiğini bilir ama kod çözücü bilmez. Bu bilgi escape sembolüyle, aritmetik kodlayıcıya bir sembol gibi gönderilir.
- Dokümanın başında sıkıştırma zayıftır
  - Yeni bir sembol (örnekte d) 9,05 bitle kodlanır; ASCII ile 8 bit olurdu. Context'ler arttıkça sıkıştırma iyileşir. LZ77 ve LZ78'de de başlangıçta çakışma olmadığı için

offset 0 olur.
- Tabloyu kendiniz kurun
  - Hoca, order-4 tablosunu kendi başınıza baştan kurup slayttakiyle karşılaştırmanızı önerdi.
- Sözlükteki string'ler anlamlı kelime değildir
  - Sözlük yöntemlerinde sözlük; bir, iki, üç ya da daha fazla karakterden oluşan, farklı uzunlukta string'ler tutar. Anlamlı kelime sözlüğü yalnız giriş örneğidir.
- Birden fazla sembolü birlikte kodlamak
  - PPM ve sözlük yöntemleri, tek tek karakter yerine daha büyük bir context'i kodladıkları için Huffman'a ve aritmetik kodlamaya göre daha iyi sıkıştırır.

Kısa Tekrar Notları

- PPM: order-N Markov, context tabanlı, adaptif; olasılıklar çıkınca aritmetik kodlama.
- Context yoksa switch: N → N-1 → … → 0 → yeni sembol ($1/$alfabe).
- Escape frekansı (PPMC) = context'i izleyen farklı sembol sayısı.
- «sa» sonrası: n 1 bit; s 2,32 bit; m 6,57 bit; d 9,05 bit.
- PPMA: escape $1/(n+1)$, diğerleri x/n. PPMB: sembol context içinde iki kez görülünce olasılık alır.
- Context'ler trie'de; derinlik order ile sınırlı; switch için pointer.
- Lipogram: olasılıkları çok ayrık metin; iyi sıkıştırılmaz.
- Statik sözlük: 20 bit (var) / 48 bit (yok); $P > 0{,}29$.
- String boyu 1, 2, 3: 1; 0,78; 0,728 bit/sembol.
- LZ77: search buffer + look-ahead; token (offset, length, sonraki); 11 + 5 + 8 = 24 bit.
- LZ77 türevleri: buffer boyları, alan uzunlukları, ilk ya da son çakışma, circular queue.
- LZ78: boş sözlük; token (index, sembol); sözlük dolunca dondur / sil / az kullanılanı değiştir.
- LZW: codebook alfabeyle başlar; «abbbaacbbabbaacacaabb» → 0 1 4 0 0 2 5 5 7 11 3 1.

Detaylı Açıklamalar

- **Order-4 tablosunun kurulması.** Tablo boşken ilk x'in geçmişi yoktur, yalnız Order 0'a x 1 yazılır. y gelince Order 0'a y 1, Order 1'e x→y 1 yazılır. z gelince Order 0, Order 1 (y→z) ve Order 2 (xy→z) güncellenir. Her yeni sembolde Order 0'dan başlayarak geçmişi 1, 2, 3, 4 olan context'ler aranır; varsa frekansı bir artırılır, yoksa 1 ile eklenir. Hoca, PPM'in aslında iki aşamada çalıştığını söyledi: önce order-N'e göre frekanslar çıkarılır, sonra bu olasılıklarla aritmetik kodlama yapılır.

- **Statik PPM'de switch.** Tablolar bir kez oluşturulduktan sonra yeni bir doküman gelir. Örneğin «kyzzx» girişinde x için Order 4'te kyzz context'i yoksa Order 3'e geçilir ve yzz→x aranır; o da yoksa Order 2, Order 1'e kadar inilir.

- **Escape olasılıklarının hesabı.** Her context bloğunun toplamı escape dahil alınır: as bloğunda $2+1=3$ (2/3, 1/3), ss bloğunda $2+1+2=5$, s bloğunda $3+2+1+3=9$, Order 0'da $4+6+1+2+1+5=19$.
  - Ders sonrası not: Hoca 01:02:41'de m senaryosundaki $2/5$'in yanlış yazıldığını, $1/2$ olması gerektiğini söyledi. Slayttaki sonuç (6,57 bit) ve tablo, a context'inin escape'i olan $2/5$'i kullanır: $-\log_2(1/2 \cdot 2/5 \cdot 1/19) \approx 6{,}57$ (python ile doğrulandı). $1/2$ ile sonuç 6,25 bit olurdu. Doğrusu $2/5$'tir.
  - Ders sonrası not: Hoca 01:05:37'de d tabloya eklenince Order 0'da escape'in 6 olacağını ve olasılıkların 20'ye bölüneceğini söyledi. Sembol frekansları $4+6+1+2+1+1=15$, escape 6 olduğundan payda büyük olasılıkla 21'dir.

- **Trie örneğinin derinliği.** Hoca order 3 için ağacın seviyesinin 3 olacağını söyledi.
  - Ders sonrası not: Slayttaki «zxzyzxxyzx» trie'lerinin derinliği 3'tür (2 sembollük context + sembol). Bu trie'ler büyük olasılıkla order-2 içindir; order-N için derinlik $N+1$ olur.

- **Lipogram tanımı.**

  - Ders sonrası not: Hoca Gadsby'de "büyük E harfi" geçmediğini söyledi. Bu romanda büyük olasılıkla e harfi küçük ya da büyük hiç kullanılmamıştır. Lipogram kelimesi genelde bir ya da birkaç harfin bilerek hiç kullanılmadığı metin için kullanılır.

- **Statik sözlük hesabındaki birim.** Hoca 01:45:59'da beş karakterlik kelime için "40 byte" dedi; $5 \times 8 = 40$ bittir. Slayttaki ara adım «9P = 8» hatalıdır; hoca tahtada $-28P + 48 = 40 \Rightarrow 28P = 8 \Rightarrow P = 8/28 \approx 0{,}2857$ olarak düzeltti (ekrandan, 01:48:30). Slayta göre özel durumlarda birkaç yüz kelimelik statik sözlükler daha iyi sonuç verir.
- **String sıkıştırma tablosu (ekrandan, 01:53:40).** Varyans, her olasılığın ortalama olasılıktan farkının mutlak değerlerinin toplamıdır. Üçlü string'lerde ortalama olasılık $1/8 = 0{,}125$; Huffman kodları 0, 100, 101, 11100, 110, 11101, 11110, 11111; ortalama $1 \cdot 0{,}512 + 3 \cdot 3 \cdot 0{,}128 + 3 \cdot 5 \cdot 0{,}032 + 5 \cdot 0{,}008 = 2{,}184$ bit/string, yani $0{,}728$ bit/sembol. İkili string'lerde ortalama $1{,}56$ bit/string, yani $0{,}78$ bit/sembol.
  - Ders sonrası not: Hoca 01:53:22'de üçlü string'ler için ortalama olasılığa "0,25" dedi; slaytta doğrusu $0{,}125$'tir.
- **LZ77'de hangi çakışma alınır?** Arama alanı sağdan sola taranır. Aynı uzunlukta birden fazla çakışma varsa bazı türevler ilk bulunanı, slayttaki yöntem en son bulunanı alır. Offset'in 0'dan mı 1'den mi sayılacağı da türevden türeve değişir; slayttaki 8 ve 16 offset'leri arama alanının sağ ucundan olan uzaklıktır.
- **LZ77 çözme.** (0,0,«s»), (0,0,«i»), (0,0,«r»), (0,0,«␣») ile «sir␣» yazılır. (4,2,«d»): 4 geriden başlayıp 2 sembol (si) alınır ve d eklenir. (4,1,«e»): 4 geriden 1 sembol (␣) alınır ve e eklenir. Çözücü yalnız indeksi yürüterek yazdığı için çok hızlıdır.
- **Circular queue örneği (hocanın sözlü örneği).** Pencerede A, B, C, D olsun; start A'da, end D'dedir. K gelince sona eklenmez, A'nın yerine yazılır; start B'ye, end K'ya geçer. J gelince B'nin yerine yazılır. Böylece kaydırma yapılmadan en eski sembol dışarı çıkar.
- **LZ78'de uzun çakışma.** Hoca, sözlükte «sir␣» bulunduğunu varsayarak «sir␣s» girişinin (25, «s») gibi tek bir token'la kodlanabileceğini söyledi: 5 byte 2 byte'lık bir token'la kodlanır. Index alanı sözlük boyuyla orantılıdır: 1 byte ile sözlükte 255 string tutulabilir.
- **LZW'de codebook boyu sınırlıysa.** Codebook dolduktan sonra yeni string eklenmez; var olan en uzun eşleşmenin kodu yazılır ve kalınan yerden devam edilir.
  - Ders sonrası not: Hoca 03:05:04'te codebook'u 8 kayıtla (0-7, son kayıt «ac») sınırlayıp örneği yeniden kodladı ve sonu "… 7, 7, 3, 4" diye anlattı; kendisi de karıştırdığını söyledi. Python ile hesaplanan çıkış 0 1 4 0 0 2 5 5 7 7 6 4'tür: «bba» (5), «bba» (5), «ac» (7), «ac» (7), «aa» (6), «bb» (4).
- **LZW'de kodların bit uzunluğu.** Codebook 14 kayıtlı (0-13) olduğu için slaytta kodlar 4 bitle (0000-1101) gösterilir. Codebook 32 elemanlı verilirse her kod 5 bitle yazılır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.